

Annotations	A	$::=$	$\overline{\text{@spec } E} \mid \varepsilon$
Expressions	E	$::=$	$x(\bar{T}) :: T \mid x(\bar{T}) :: T \text{ when } \overline{\alpha: G}$
Guards	G	$::=$	$T \mid \text{var}$
Types	T	$::=$	$\alpha \mid T \mid T$ $\mid \text{any}() \mid \text{none}()$ $\mid \text{pid}() \mid \text{port}() \mid \text{reference}()$ $\mid \text{atom}() \mid k$ $\mid \text{float}() \mid \text{integer}()$ $\mid \text{neg_integer}() \mid \text{non_neg_integer}() \mid \text{pos_integer}()$ $\mid \% \{ \bar{F} \}$ $\mid \text{tuple}() \mid \{ \bar{T} \}$ $\mid \text{list}(T) \mid \text{nonempty_list}(T)$ $\mid \text{maybe_improper_list}(T, T)$ $\mid \text{nonempty_improper_list}(T, T)$ $\mid \text{nonempty_maybe_improper_list}(T, T)$ $\mid \text{«} \mid \text{«_::}n \text{»} \mid \text{«_::_}n \text{»} \mid \text{«_::}n, _::_n \text{»}$ $\mid (\bar{T} \rightarrow T) \mid (\dots \rightarrow T)$ $\mid n \mid n..n'$
Fields	F	$::=$	$\text{required}(T) \Rightarrow T \mid \text{optional}(T) \Rightarrow T$

(where x ranges over expression variables, α ranges over type variables, n ranges over integers with $n < n'$, and k ranges over atoms)

Figure 1: TypeSpec

Annotations	A	$::=$	$\$ E \mid \varepsilon$
Expressions	E	$::=$	$\bar{T} \rightarrow T \mid \bar{T} \rightarrow T \text{ when } \overline{\alpha: T}$
Types	T	$::=$	$\alpha \mid T \text{ or } T \mid T \text{ and } T \mid \text{not } T$ $\mid K \mid \text{term}() \mid \text{none}() \mid \text{dynamic}()$ $\mid \% \{ \bar{F} \} \mid \% \{ \dots, \bar{F} \}$ $\mid \{ \bar{T} \} \mid \{ \bar{T}, \dots \}$ $\mid \text{empty_list}() \mid \text{nonempty_list}(T, T)$ $\mid (\bar{T} \rightarrow T)$
Key types	K	$::=$	$\text{pid}() \mid \text{port}() \mid \text{reference}()$ $\mid \text{binary}() \mid \text{atom}() \mid k$ $\mid \text{float}() \mid \text{integer}() \mid n..n$
Fields	F	$::=$	$k \Rightarrow T \mid K \Rightarrow \text{is_set}(T)$

(where x ranges over expression variables, α ranges over type variables, $n..n$ ranges over integers invisible to the users, and k ranges over atoms)

Figure 2: Elixir Type

1 Basic Types

1. Top types

- `any() == var == term()`
=> Compatible in the type migration as Elixir Type has `term()`

2. Bottom type

- `none()`
=> Compatible in the type migration

3. Identifier types

- `pid()`
A pid (process identifier) refers into a process table and a node table, which also consumes memory.
1 word for a pid from the current local node.
- `port()`
A port identifier refers into a port table and a node table, which also consumes memory.
- `reference()`
A reference also refers into more or less emulator internal data structures which also consumes memory. At a minimum it refers into the node tables.
=> All compatible in the type migration

4. Atom types

- `atom()` An atom, `'k'`, can be used as a type while `'true'`, `'false'`, `'nil'` are reserved atoms that are used without prepositional `':'`.
=> Compatible in the type migration

5. Number types

- `float()`
=> Compatible in the type migration
- `integer()`
TypeSpec's `integer()` has subtypes that are `neg_integer()`, `non_neg_integer()`, and `pos_integer()`. Furthermore, an individual integer, `'n'`, can be used as a type while it supports range of integer with `'..'` operator. Whereas, in Elixir type, `integer()` is indivisible.
However, the future work could either split `integer()` into: individual elements; ranges; or `neg_integer() | zero() | pos_integer()`.
=> Subtypes of `integer()` are incompatible in the type migration

6. Map types

- `map()`
 - TypeSpec Syntactic Sugars
 - (a) Empty map:
 $\%{\}$
 - (b) Map with required key ‘: *key*’ of value_type:
 $\%{\text{key: value_type}} == \%{\text{required(atom())} \Rightarrow \mathbf{T}}$
 - (c) Map with required pairs of key_type and value_type:
 $\%{\text{key_type} \Rightarrow \text{value_type}}$
 $== \%{\text{required(key_type)} \Rightarrow \text{value_type}}$
 $== \%{\text{required}(\mathbf{T}) \Rightarrow \mathbf{T}}$
 - (d) Map with optional pairs of key_type and value_type
 $\%{\text{optional(key_type)} \Rightarrow \text{value_type}}$
 $== \%{\text{optional}(\mathbf{T}) \Rightarrow \mathbf{T}}$
 - (e) Struct with all fields of any type
 $\%{\text{SomeStruct}\{\}} == \%{\text{required}(\text{:__struct_}) \Rightarrow x}$
 - (f) Struct with required key :key of value_type
 $\%{\text{SomeStruct}\{\text{key: value_type}\}}$
 $== \%{\text{required}(\text{:__struct_}) \Rightarrow x \text{ required}(k) \Rightarrow \mathbf{T}}$
- $\Rightarrow$ Compatible in the type migration

7. List types

- Lists of types
 $\text{list}(\text{type}) \Rightarrow \text{empty_list}() \text{ or } \text{non_empty_list}(\mathbf{T})$
- Non-empty lists of types
 $\text{non_empty_list}(\text{type})$
 $\Rightarrow \text{non_empty_list}(\mathbf{T})$
 $== \text{non_empty_list}(\mathbf{T}, \text{empty_list}())$
- Proper or improper lists
 $\text{maybe_improper_list}(\text{content_type}, \text{termination_type})$
 $\Rightarrow \text{list}(\mathbf{T}) \text{ or } \text{non_empty_list}(\mathbf{T}, \mathbf{T} \text{ and not } \text{list}())$
- Improper lists
 $\text{nonempty_improper_list}(\text{content_type}, \text{termination_type})$

$\Rightarrow \text{non_empty_list}(T, T \text{ and not list}())$

- Non-empty proper or improper lists
 $\text{nonempty_maybe_improper_list}(\text{content_type}, \text{termination_type})$
 $\Rightarrow \text{non_empty_list}(T) \text{ or } \text{non_empty_list}(T, T \text{ and not list}())$

Expressions	$E ::= L \mid x \mid A \text{ fn } \overline{P} \text{ when } G \rightarrow E \text{ end} \mid E (E_1, \dots, E_n) \mid E + E$ $\mid \text{case } E \text{ } \overline{P} \text{ when } G \rightarrow \overline{E} \mid \{E_1, \dots, E_n\} \mid \text{elem}(E, E)$
Singletons	$L ::= n \mid k \mid \{\overline{L}\}$
Patterns	$P ::= L \mid x \mid \{P_1, \dots, P_n\}$
Guards	$G ::= D \mid C \mid \text{not } G \mid G \text{ and } G \mid G \text{ or } G \mid G == G \mid G != G$ $\mid G < G \mid G \leq G \mid G > G \mid G \geq G$
Selectors	$D ::= L \mid x \mid \text{elem}(D, D) \mid \text{tuple_size}(D) \mid \{\overline{D}\}$
Checks	$C ::= \text{is_integer}(D) \mid \text{is_atom}(D) \mid \text{is_tuple}(D)$ $\mid \text{is_function}(D) \mid \text{is_function}(D, n)$
Base types	$B ::= \text{integer}() \mid \text{atom}() \mid \text{function}() \mid \text{tuple}()$
Types	$T ::= B \mid L \mid (\overline{T}) \rightarrow T \mid \{\overline{T}\} \mid \{\overline{T}, \dots\} \mid T \text{ or } T \mid T \text{ and } T \mid \text{not } T$ $\mid \text{none}() \mid \text{term}() \mid \text{dynamic}()$
Annotations	$A ::= \$ T \mid \varepsilon$

(where x ranges over variables, n ranges over integers, and k ranges over atoms)

Figure 3: Featherweight Elixir